

TECHNICAL EXPLAINER · NOT A PITCH DOCUMENT

How POLAR-EMS actually works, end to end

What the system is for, what runs inside it every second it's alive, and how that combination measurably accomplishes its goal — keeping an isolated polar research station powered safely while burning less diesel.

Live site polar-ems-web.vercel.app **Backend** polar-ems-backend.onrender.com

Source github.com/sayansaunth-lang/polar-ems-sih

01 Purpose — the goal task

POLAR-EMS exists to solve one concrete operational problem for an isolated Antarctic research station, then prove — with numbers, not claims — that it solves it better than the alternative.

A polar station's power system is a self-contained microgrid: wind turbines, solar panels, a battery bank, and a diesel generator as the fallback of last resort. There is no utility grid to fall back on. Every litre of diesel arrived by ship or aircraft at enormous cost, satellite internet can vanish for weeks at a time, and losing heating or life-support power in a -40°C environment is a safety event, not an inconvenience.

The goal task, precisely stated: **continuously decide, tick by tick, how much power to draw from wind, solar, battery and diesel to meet station load — minimizing diesel burned and generator runtime — while guaranteeing that critical loads (life support, heating) never lose power, even if the AI, the sensors, or the internet link fail.**

Why this needs to be measured, not asserted:	it would be easy to build a dashboard that simply claims to save fuel. POLAR-EMS instead runs two independent dispatch controllers — a fixed-rule baseline and a forecast-aware optimizer — on	<i>identical</i> weather and load conditions every single tick, so the fuel/ CO_2 delta shown on the Green Audit tab is a measured difference between two real running controllers, not a hardcoded number.
---	--	--

02 System architecture at a glance

Five layers, each one able to keep operating if everything above it disappears — the architecture itself is the safety argument.

01	Field sensors	Weather, battery, generator and per-zone load telemetry.	Simulated here from a physical model
02	Edge gateway	Ingests and stores telemetry locally; runs every model below entirely offline.	Survives total internet loss
03	AI engine	Load/renewable forecasting, dispatch optimization, anomaly detection.	Survives sparse/noisy data
04	Dispatch & safety	Executes source selection and load shedding; falls back to hard-coded rules if the AI layer is unavailable.	Survives AI/model failure
05	Cloud portal	Historical analytics and green-audit reporting, synced in batches whenever a link exists.	Fine offline for weeks

Two independent implementations of the same model

The **frontend** (plain HTML/CSS/JavaScript, no framework, no build step) runs the entire simulation client-side — weather, dispatch, safety, forecasting, charts — so the dashboard works with zero backend at all, proving the "edge autonomy" story for real. The **backend** (Python, FastAPI) is a second, independently-runnable implementation of the same physical model, exposing it as a REST API with interactive docs, plus a terminal CLI client. Either can be demoed alone; an optional "Live Backend" panel on the site can also poll the real server to show the two agreeing.

03 What happens every simulated tick

The entire simulation is one function, `tick()`, that runs on a timer and does the following, in this exact order, entirely in the browser:

1 Advance simulated time & generate this tick's weather

`stepWeather()` — a seasonal solar/wind model (polar day/night cycle, turbine power curve) plus autoregressive noise, so weather is physically plausible and continuous, not random per tick.

2 Compute renewable generation & station load

`computeRenewables()` applies the turbine power curve and solar irradiance model; `computeLoad()` sums four priority tiers (life support/heating, research, general, amenities) with realistic diurnal variation.

3 Run *both* dispatch controllers on identical inputs

`dispatchBaseline()` (fixed rules) and `dispatchAI()` (forecast-aware optimizer) each decide the wind/solar/battery/diesel mix independently, from the exact same weather and load — this is what makes the fuel-savings comparison honest.

4 Check whether the AI is even allowed to run

If a simulated AI-service failure or sensor fault is active, the tick silently substitutes `dispatchBaseline()` in place of `dispatchAI()` — a genuine controller swap, not just a relabeled badge.

5 Apply the hard safety override

`applySafety()` sits outside and after the optimizer's decision. If battery state-of-charge would fall below the critical floor, it forces diesel on and sheds non-critical load — regardless of what the optimizer wanted. This function cannot be overridden by the AI.

6 Train the online ML forecaster, one step

`mlTrainStep()` — a real gradient-descent update to an online linear regression, using this tick's actual load as the training signal. It never stops learning.

7 Generate & resolve forecasts

`generateForecasts()` produces 1/6/12/24-hour-ahead predictions from three independent models (statistical, online ML, deployed pretrained ML — see next section); `resolveForecasts()` scores each one's past predictions against what actually happened, updating live MAE/RMSE.

8 Check for anomalies

`checkAnomalies()` compares each subsystem's actual reading against what the physical model says it *should* be, and only fires an alert when the gap crosses a statistical threshold — every alert states its expected-vs-detected values, never "the model said so."

9 Record history & render

This tick's full state is pushed into a rolling history buffer (fuel used, SOC, mode, decision reason, cumulative audit totals), and every visible chart, KPI and 3D readout re-renders from that same state.

One tick,
one
source of
truth:

every number on every tab, plus the cinematic 3D hero and the digital twin diagram, is read from this same tick's output through a single adapter (

window.PolarEMS

)

There is no second, decorative data source anywhere on the page.

04 Two controllers, one goal: measuring the value

This is the mechanism that actually accomplishes the goal task — not a claim, a running comparison.

BASELINE CONTROLLER	AI-OPTIMIZED CONTROLLER
Fixed rules: use renewables first, discharge battery on a fixed schedule, start diesel only once SOC crosses a fixed threshold.	Same physical constraints, but timing informed by the load/renewable forecast — e.g. pre-emptively holding more battery reserve ahead of a forecast low-wind spell, instead of reacting only once it arrives.
Sensible and safe on its own — not a strawman.	Wins only because it looks ahead, not because the baseline is broken.

Both run every tick on identical conditions. The Green Audit tab reports the accumulated difference directly: **fuel saved** (litre delta between the two controllers' cumulative diesel burn), **CO₂ avoided** (that delta × 2.68 kg/L, the standard diesel emission factor), and **generator runtime reduction** (hours). That delta is what "AI-driven" is actually claiming credit for — a forecast-aware policy beating a fixed rule at the same job, under the same conditions, measured continuously.

05 Where the AI actually is

Specific and honest about which parts are learned and which are deliberately not, because in a safety-relevant system that distinction is the point.

DEPLOYED PRETRAINED

Gradient-boosting regressor, running inference in the browser

Scikit-learn `GradientBoostingRegressor` (150 trees), trained offline on a genuine held-out train/test split (5.4 kW mean error on unseen data). Its trees were exported to a JSON file and re-implemented as a small JavaScript tree-walking function — verified to reproduce scikit-learn's own predictions bit-for-bit before shipping. Runs entirely client-side; no server call at prediction time.

DEPLOYED PRETRAINED

The same model, server-side

Identical trained model also runs inside the FastAPI backend, exposed at `/api/forecast/ml`, so the same trained weights are reachable from a real HTTP API and a terminal CLI, not only the browser.

LEARNS LIVE

Online linear regression (Widrow-Hoff / LMS)

Trains itself one gradient step per simulated tick, directly from this session's own telemetry — no offline dataset. Its weights and live MAE/RMSE are visible on the Forecast tab and genuinely change as the simulation runs.

DELIBERATELY NOT AI

Dispatch optimizer & hard safety override

A constrained, explainable decision policy and a hard-coded safety floor — not learned. In a system where a wrong decision can mean losing heat, "the model decided" is not an acceptable answer for why power was cut; a policy you can fully audit is more trustworthy than a black box here.

DELIBERATELY NOT AI

Anomaly detection

Statistical control limits — actual reading vs. physically-expected value, threshold-triggered. Every alert names both numbers, so "how do you know it's anomalous" always has a specific answer.

06 Safety & offline autonomy

The part of the architecture that exists specifically because failure here is a safety event, not an inconvenience.

- **Hard safety floor:** if battery SOC drops toward the critical threshold, `applySafety()` forces diesel on and sheds only the lowest-priority loads — life support and heating sit in an unshed tier, enforced independently of any AI decision.
- **Graceful AI failure:** triggering the "AI service failure" scenario genuinely swaps the running controller to the deterministic baseline — visible live as the mode badge flips to SAFE FALLBACK — rather than just relabeling the same logic.
- **Offline autonomy:** toggling a simulated internet outage flips a real state flag; telemetry, forecasting, optimization and the safety layer all keep running unchanged. Only the cloud sync pauses, queueing records into a local outbox that drains with an animated batch upload once connectivity returns — mirroring real store-and-forward behavior on an intermittent satellite link.

07 The cinematic 3D layer & digital twin

A Three.js hero scene and a live digital-twin flow diagram sit on top of the dashboard — built specifically so they can never drift from the real simulation.

Both read the tick loop's output through one read-only adapter, `window.PolarEMS`, exposing a `snapshot()` of the current state and an `onTick()` subscription. If the Dashboard tab says battery is at 72%, the 3D hero's HUD and the digital twin's battery node show the same 72%, because they're reading the identical number, not a separate illustration. Clicking a subsystem in the digital twin (wind, solar, battery, diesel) both updates its live info panel and moves the 3D camera to focus on that subsystem's model — GPU-tiered (HIGH/MEDIUM/LOW geometry detail) so it degrades gracefully on lower-end hardware, and recomposed (not just resized) for narrow mobile viewpoints.

A looping background video of the station sits fixed behind the entire scroll as ambient atmosphere — purely decorative, contributing no data of its own, and removed outright under `prefers-reduced-motion`

08 Backend & deployment

PIECE	WHAT IT IS	WHERE IT RUNS
Frontend	Single-file HTML/CSS/JS, no build step, no framework	Vercel (static CDN)
Backend	Python + FastAPI, the same physical model reimplemented server-side	Render (containerized, HTTPS, auto-deploys on push)
CLI client	<code>cli.py</code> — a terminal menu that only ever makes plain HTTP calls to the API, no logic of its own	Runs locally against either backend
Source control	Git	GitHub

A real constraint we hit and fixed, not hidden: browsers refuse to let an HTTPS page call a plain HTTP address ("mixed content") — encountered directly when the frontend on Vercel (HTTPS) tried to reach a local backend (HTTP). Fixed by giving the backend real HTTPS on Render, not by bypassing browser security.

09 How this accomplishes the goal task

Tying it back to section 01.

1. **Minimizes diesel burned:** proven continuously by the baseline-vs-AI comparison, not asserted — the Green Audit tab is a running measurement, not a static claim.
2. **Never loses critical power:** the hard safety layer is architecturally outside the optimizer's reach, and its behavior under AI failure is demonstrable live, on demand.
3. **Keeps working with zero connectivity:** every layer that matters — sensing, forecasting, optimizing, safety — runs at the edge; only historical reporting needs the cloud, and it tolerates being offline for as long as necessary.
4. **Is explainable, not a black box:** every dispatch decision carries a plain-English reason, every anomaly alert names its expected-vs-detected comparison, and every "trained" claim points at a specific model with a specific measured accuracy — visible on the Forecast tab, right now, while it runs.

